3. 프로그램 구조 및 AST 모듈 분석

3.1 핵심 AST 노드 구조 분석

Circom 컴파일러의 `program_structure` 모듈은 추상 구문 트리(AST)의 핵심 데이터 구조를 정의하고 관리합니다. 이 모듈은 `.circom` 파일의 구문 요소들을 메모리 내 표현으로 변환하고, 후속 컴파일 단계에서 활용할 수 있는 구조화된 데이터를 제공합니다.

AST 최상위 구조

AST 루트 노드

```rust
#[derive(Clone)]
pub struct AST {
pub meta: Meta,
pub compiler_version: Option<Version>,
pub custom_gates: bool,
pub custom_gates_declared: bool,
pub includes: Vec<String>,
pub definitions: Vec<Definition>,
pub main_component: Option<MainComponent>,
}
```

`AST` 구조체는 전체 Circom 프로그램의 최상위 표현으로, 다음 요소들을 포함합니다:

- **meta**: 파일 위치 및 메타데이터 정보

- **compiler_version**: pragma 지시문으로 지정된 컴파일러 버전
- **custom_gates**: 커스텀 게이트 사용 여부
- **includes**: 포함된 파일 목록
- **definitions**: 템플릿, 함수, 버스 정의들
- **main_component**: 메인 컴포넌트 정의

핵심 정의(Definition) 노드들

Definition 열거형

```
#[derive(Clone)]
pub  enum Definition {
Template {
meta: Meta,
name: String,
args: Vec<String>,
arg_location: FileLocation,
body: Statement,
parallel: bool,
is_custom_gate: bool,
is_extern_c: bool
},
Function {
meta: Meta,
name: String,
```

```
args: Vec<String>,

arg_location: FileLocation,

body: Statement,

},

Bus {

meta: Meta,

name: String,

args: Vec<String>,

arg_location: FileLocation,

body: Statement,

},

}
```

Template 노드 상세 분석

Template은 Circom의 핵심 구성 요소로, 재사용 가능한 회로 블록을 정의합니다:

- **name**: 템플릿 식별자
- **args**: 매개변수 목록 (템플릿 인스턴스화 시 사용)
- **arg_location**: 매개변수 정의 위치 정보
- **body**: 템플릿 본문 (Statement 형태)
- **parallel**: 병렬 처리 가능 여부
- **is_custom_gate**: 커스텀 게이트 여부
- **is_extern_c**: 외부 C 함수 연동 여부

Function과 Bus 노드

Function은 순수 계산 함수를, **Bus**는 신호 그룹을 나타내며, 모두 유사한 구조를 가지지만 의미적으로 다른 역할을 수행합니다.

Statement 노드 구조

Statement 열거형

```
#[derive(Clone)]
pub  enum Statement {
IfThenElse {
meta: Meta,
cond: Expression,
if_case: Box<Statement>,
else_case: Option<Box<Statement>>,
},
While {
meta: Meta,
cond: Expression,
stmt: Box<Statement>,
},
Return {
meta: Meta,
value: Expression,
},
```

```
InitializationBlock {
meta: Meta,
xtype: VariableType,
initializations: Vec<Statement>,
},
Declaration {
meta: Meta,
xtype: VariableType,
name: String,
dimensions: Vec<Expression>,
is_constant: bool,
is_anonymous: bool
},
Substitution {
meta: Meta,
var: String,
access: Vec<Access>,
op: AssignOp,
rhe: Expression,
},
MultSubstitution {
meta: Meta,
lhe: Expression,
```

```
op: AssignOp,
rhe: Expression,
},
UnderscoreSubstitution{
meta: Meta,
op: AssignOp,
rhe: Expression,
},
ConstraintEquality {
meta: Meta,
lhe: Expression,
rhe: Expression,
},
LogCall {
meta: Meta,
args: Vec<LogArgument>,
},
Block {
meta: Meta,
stmts: Vec<Statement>,
},
Assert {
meta: Meta,
```

```
arg: Expression,
},
}
```

주요 Statement 노드 분석

Declaration: 변수, 신호, 컴포넌트 선언

- `xtype: VariableType` 열거형으로 신호 타입 구분
- `dimensions` : 배열 차원 정보
- `is_constant` : 상수 여부
- `is_anonymous` : 익명 컴포넌트 여부

ConstraintEquality: R1CS 제약 조건 생성의 핵심

- `lhe` , `rhe` : 좌변과 우변 표현식
- 컴파일 시 산술 제약 조건으로 변환됨

Substitution: 변수 할당 연산

- `access` : 배열/컴포넌트 접근 경로
- `op` : 할당 연산자 타입 (`AssignOp`)

Expression 노드 구조

Expression 열거형

```
#[derive(Clone)]
pub enum Expression {
InfixOp {
meta: Meta,
```

```
lhe: Box<Expression>,
infix_op: ExpressionInfixOpcode,
rhe: Box<Expression>,
},
PrefixOp {
meta: Meta,
prefix_op: ExpressionPrefixOpcode,
rhe: Box<Expression>,
},
InlineSwitchOp {
meta: Meta,
cond: Box<Expression>,
if_true: Box<Expression>,
if_false: Box<Expression>,
},
ParallelOp {
meta: Meta,
rhe: Box<Expression>,
},
Variable {
meta: Meta,
name: String,
access: Vec<Access>,
```

```
},
Number(Meta, BigInt),
Call {
meta: Meta,
id: String,
args: Vec<Expression>,
},
BusCall {
meta: Meta,
id: String,
args: Vec<Expression>,
},
AnonymousComp{
meta: Meta,
id: String,
is_parallel: bool,
params: Vec<Expression>,
signals: Vec<Expression>,
names: Option<Vec<(AssignOp, String)>>,
},
ArrayInLine {
meta: Meta,
values: Vec<Expression>,
```

```
},
Tuple {
meta: Meta,
values: Vec<Expression>,
},
UniformArray {
meta: Meta,
value: Box<Expression>,
dimension: Box<Expression>,
},
}
```

핵심 Expression 노드 분석

Variable: 변수 참조

- `access` : 배열 인덱스나 컴포넌트 필드 접근 경로
- `Access` 열거형: `ComponentAccess(String)` 또는 `ArrayAccess(Expression)`

AnonymousComp: 익명 컴포넌트 인스턴스화

- `params` : 템플릿 매개변수
- `signals` : 신호 연결 정보
- `names` : 신호 이름 매핑

InfixOp/PrefixOp: 산술 및 논리 연산

- 컴파일 시 R1CS 제약 조건으로 변환되는 핵심 요소

변수 타입 시스템

VariableType과 SignalType

```
#[derive(Copy, Clone, PartialEq, Eq, PartialOrd, Ord, Serialize, Deserialize)]
pub enum SignalType {
Output,
Input,
Intermediate,
}

#[derive(Clone, PartialEq, Eq)]
pub enum VariableType {
Var,
Signal(SignalType, TagList),
Component,
AnonymousComponent,
Bus(String, SignalType, TagList),
}
```

SignalType은 zkSNARK 회로의 신호 분류를 나타냅니다:

- **Input**: 회로 입력 (공개 또는 비밀)
- **Output**: 회로 출력
- **Intermediate**: 중간 계산 결과

VariableType은 변수의 의미적 분류를 제공하며, 타입 검사와 제약 조건 생성에 활용됩니다.

메타데이터 및 위치 정보 처리

Meta 구조체

```
#[derive(Clone)]
pub struct Meta {
pub elem_id: usize,
pub start: usize,
pub end: usize,
pub location: FileLocation,
pub file_id: Option<usize>,
pub component_inference: Option<String>,
type_knowledge: TypeKnowledge,
memory_knowledge: MemoryKnowledge,
}
```

Meta는 모든 AST 노드에 포함되는 메타데이터로:

- **elem_id**: 고유 요소 식별자
- **location**: 소스 코드 위치 (`start..end` 범위)
- **file_id**: 파일 식별자
- **type_knowledge**: 타입 추론 정보
- **memory_knowledge**: 메모리 레이아웃 정보

TypeKnowledge와 MemoryKnowledge

```rust
#[derive(Default, Clone)]
pub struct TypeKnowledge {
reduces_to: Option<TypeReduction>,
}

#[derive(Default, Clone)]
pub struct MemoryKnowledge {
concrete_dimensions: Option<Vec<usize>>,
full_length: Option<usize>,
abstract_memory_address: Option<usize>,
}
```

이들은 타입 분석과 메모리 할당 단계에서 점진적으로 채워지는 정보를 저장합니다.

AST 순회 및 방문자 패턴 구현

FillMeta 트레이트

```rust
pub trait FillMeta {
fn fill(&mut self, file_id: usize, elem_id: &mut usize);
}
```

FillMeta 트레이트는 AST 노드에 메타데이터를 채우는 방문자 패턴을 구현합니다. 모든 주요 AST 노드(`Expression` , `Statement`)가 이 트레이트를 구현하여:

1. **elem_id** 할당: 각 노드에 고유 식별자 부여

2. **file_id** 설정: 소스 파일 정보 연결

3. **재귀적 순회**: 자식 노드들에 대해 동일한 작업 수행

Expression의 FillMeta 구현 예시

```
impl FillMeta for Expression {
fn fill(&mut self, file_id: usize, element_id: &mut usize) {
self.get_mut_meta().elem_id = *element_id;
*element_id += 1;
match self {
InfixOp { lhe, rhe, .. } => {
lhe.fill(file_id, element_id);
rhe.fill(file_id, element_id);
}
Variable { access, .. } => {
for acc in access {
if let Access::ArrayAccess(e) = acc {
e.fill(file_id, element_id)
}
}
}
// ... 다른 Expression 변형들
}
}
```

```
}
```

AST 노드 검사 메서드들

각 AST 노드는 타입 검사를 위한 편의 메서드들을 제공합니다:

Expression 검사 메서드들:

```
impl Expression {
pub fn is_variable(&self) -> bool { /* ... */ }
pub fn is_call(&self) -> bool { /* ... */ }
pub fn is_anonymous_comp(&self) -> bool { /* ... */ }
pub fn contains_anonymous_comp(&self) -> bool { /* ... */ }
pub fn contains_tuple(&self) -> bool { /* ... */ }
}
```

Statement 검사 메서드들:

```
impl Statement {
pub fn is_declaration(&self) -> bool { /* ... */ }
pub fn is_constraint_equality(&self) -> bool { /* ... */ }
pub fn contains_anonymous_comp(&self) -> bool { /* ... */ }
}
```

이러한 메서드들은 타입 분석, 제약 조건 생성, 최적화 단계에서 특정 패턴의 노드를 식별하고 처리하는 데 활용됩니다.

AST 구조의 설계 원칙

1. **불변성과 복제**: 모든 AST 노드는 `Clone` 트레이트를 구현하여 안전한 복사가 가능합니다.
2. **메타데이터 일관성**: 모든 노드가 `Meta` 정보를 포함하여 에러 리포팅과 디버깅을 지원합니다.
3. **타입 안전성**: Rust의 열거형과 패턴 매칭을 활용하여 컴파일 타임 타입 안전성을 보장합니다.
4. **확장성**: 새로운 언어 기능 추가 시 기존 구조를 깨뜨리지 않고 확장할 수 있는 설계입니다.

이러한 AST 구조는 Circom 언어의 모든 구문 요소를 정확히 표현하면서도, 후속 컴파일 단계에서 효율적으로 처리할 수 있는 기반을 제공합니다.

3.

---

2 프로그램 아카이브 구조 분석

`ProgramArchive` 는 Circom 컴파일러에서 다중 파일 프로젝트를 통합 관리하는 핵심 데이터 구조입니다. 파싱된 개별 파일들의 AST를 하나의 일관된 프로그램 표현으로 병합하고, 후속 컴파일 단계에서 필요한 모든 정보를 제공합니다.

ProgramArchive 핵심 구조

ProgramArchive 정의

```
#[derive(Clone)]
pub struct ProgramArchive {
pub id_max: usize,
pub file_id_main: FileID,
pub file_library: FileLibrary,
pub functions: FunctionInfo,
pub templates: TemplateInfo,
pub buses: BusInfo,
pub function_keys: HashSet<String>,
```

```
pub template_keys: HashSet<String>,
pub bus_keys: HashSet<String>,
pub public_inputs: Vec<String>,
pub initial_template_call: Expression,
pub custom_gates: bool,
}
```

ProgramArchive의 주요 구성 요소:

- **id_max**: 전체 프로그램에서 사용된 최대 요소 ID (고유 식별자 생성용)
- **file_id_main**: 메인 파일의 식별자
- **file_library**: 모든 소스 파일의 내용과 메타데이터 저장소
- **functions/templates/buses**: 각 타입별 정의들의 해시맵
- **function_keys/template_keys/bus_keys**: 빠른 검색을 위한 이름 집합
- **public_inputs**: 메인 컴포넌트의 공개 입력 목록
- **initial_template_call**: 메인 컴포넌트 인스턴스화 표현식
- **custom_gates**: 커스텀 게이트 사용 여부

파일 라이브러리 관리 시스템

FileLibrary 구조

```
#[derive(Clone)]
pub struct FileLibrary {
files: FileStorage,
}
```

```
type FileStorage = SimpleFiles<FilePath, FileSource>;

type FilePath = String;

type FileSource = String;

type FileID = usize;

type FileLocation = Range<usize>;
```

FileLibrary는 `codespan_reporting` 크레이트의 `SimpleFiles` 를 기반으로 구현되어:

1. **파일 저장**: 각 파일의 경로와 소스 코드를 저장
2. **위치 추적**: 소스 코드 내 위치 정보를 파일별로 관리
3. **에러 리포팅**: 컴파일 에러 발생 시 정확한 소스 위치 제공

파일 관리 주요 메서드

```
impl FileLibrary {

pub  fn  add_file(&mut  self, file_name: FilePath, file_source: FileSource) -> F

self.get_mut_files().add(file_name, file_source)

}

pub  fn  get_line(&self, start: usize, file_id: FileID) -> Option<usize> {

match  self.files.line_index(file_id, start) {

Some(lines) => Some(lines + 1),

None => None,

}

}
```

```
}
```

프로그램 병합 메커니즘 (Merger)

Merger 구조체

```
pub struct Merger {
fresh_id: usize,
function_info: FunctionInfo,
template_info: TemplateInfo,
bus_info: BusInfo,
}
```

Merger는 여러 파일의 정의들을 하나의 프로그램으로 통합하는 역할을 담당합니다:

정의 추가 과정 (`add_definitions`)

```
pub fn add_definitions(&mut self, file_id: FileID, definitions: Vec<Definitio
-> Result<(), Vec<Report>> {
let mut reports = vec![];
for definition in definitions {
match definition {
Definition::Template { name, args, body, meta, parallel, is_custom_gate, is_exte
if self.contains_template(&name) || self.contains_function(&name) || self.conta
// 중복 정의 에러 생성
let mut report = Report::error(
```

```
String::from("Duplicated callable symbol"),
ReportCode::SameSymbolDeclaredTwice,
);
report.add_primary(meta.file_location(), file_id,
format!("{} is already in use", name));
reports.push(report);
} else {
// 새 템플릿 데이터 생성 및 추가
let  template_data = TemplateData::new(
name.clone(), file_id, body, args.len(), args,
arg_location, &mut  self.fresh_id, parallel, is_custom_gate, is_extern_c
);
self.template_info.insert(name, template_data);
}
}
// Function과 Bus에 대해서도 유사한 처리
}
}
}
```

병합 과정의 핵심 기능:

1. **중복 검사**: 동일한 이름의 템플릿/함수/버스가 이미 존재하는지 확인
2. **메타데이터 채우기**: `FillMeta` 트레이트를 통해 각 AST 노드에 파일 정보 설정

3. **고유 ID 할당**: 전체 프로그램에서 유일한 요소 ID 부여

4. **에러 수집**: 중복 정의 등의 문제를 Report로 수집

템플릿 데이터 관리

TemplateData 구조

```
#[derive(Clone)]
pub  struct TemplateData {
file_id: FileID,
name: String,
body: Statement,
num_of_params: usize,
name_of_params: Vec<String>,
param_location: FileLocation,
input_wires: WireInfo,
output_wires: WireInfo,
is_parallel: bool,
is_custom_gate: bool,
is_extern_c: bool,
input_declarations: WireDeclarationOrder,
output_declarations: WireDeclarationOrder,
}
```

TemplateData는 템플릿의 모든 정보를 캡슐화합니다:

- **기본 정보**: 이름, 매개변수, 본문
- **신호 정보**: 입력/출력 신호의 타입과 차원
- **속성 플래그**: 병렬 처리, 커스텀 게이트, 외부 C 연동 여부
- **선언 순서**: 신호 선언 순서 (코드 생성 시 중요)

신호 정보 추출 (`fill_inputs_and_outputs`)

```
fn fill_inputs_and_outputs(
template_statement: &Statement,
input_wires: &mut WireInfo,
output_wires: &mut WireInfo,
input_declarations: &mut WireDeclarationOrder,
output_declarations: &mut WireDeclarationOrder
) {
match template_statement {
Statement::Declaration { xtype, name, dimensions, .. } => {
match xtype {
VariableType::Signal(SignalType::Input, tag_list) => {
let wire_data = WireData::new(WireType::Signal, dimensions.len(), tag_info);
input_wires.insert(name.clone(), wire_data);
input_declarations.push((name.clone(), dimensions.len()));
}
VariableType::Signal(SignalType::Output, tag_list) => {
let wire_data = WireData::new(WireType::Signal, dimensions.len(), tag_info);
```

```
output_wires.insert(name.clone(), wire_data);
output_declarations.push((name.clone(), dimensions.len()));
}
// 중간 신호는 무시 (템플릿 인터페이스에 포함되지 않음)
}
}
Statement::Block { stmts, .. } => {
// 재귀적으로 모든 하위 문장 처리
for stmt in stmts {
fill_inputs_and_outputs(stmt, input_wires, output_wires,
input_declarations, output_declarations);
}
}
// 다른 문장 타입들도 재귀적 처리
}
}
```

신호 및 와이어 데이터 관리

WireData와 WireInfo

```
#[derive(Clone)]
pub struct WireData {
wire_type: WireType,
```

```
dimension: usize,
tag_info: TagInfo,
}

#[derive(Clone, PartialEq, Eq)]
pub enum WireType {
Signal,
Bus(String),
}

pub type WireInfo = HashMap<String, WireData>;
pub type TagInfo = HashSet<String>;
pub type WireDeclarationOrder = Vec<(String, usize)>;
```

신호 관리 시스템의 특징:

1. **타입 구분**: 일반 신호와 버스 신호 구분
2. **차원 정보**: 배열 신호의 차원 수 저장
3. **태그 시스템**: `public`, `private` 등의 속성 관리
4. **선언 순서**: 원본 코드의 선언 순서 보존 (중요한 의미적 정보)

메인 컴포넌트 식별 과정

MainComponent 처리

```
pub type MainComponent = (Vec<String>, Expression);
```

```
// ProgramArchive 생성 시
let (public_inputs, mut initial_template_call) = main_component;
initial_template_call.fill(file_id_main, &mut fresh_id);
```

메인 컴포넌트 관리:

1. **공개 입력 목록**: zkSNARK 증명에서 공개될 입력들
2. **인스턴스화 표현식**: 메인 템플릿을 호출하는 Expression
3. **메타데이터 설정**: 메인 파일 정보로 메타데이터 채우기

다중 파일 프로젝트 관리 메커니즘

ProgramArchive 생성 과정

```
impl ProgramArchive {
pub fn new(
file_library: FileLibrary,
file_id_main: FileID,
main_component: MainComponent,
program_contents: Contents,
custom_gates: bool,
) -> Result<ProgramArchive, (FileLibrary, Vec<Report>)> {
let mut merger = Merger::new();
let mut reports = vec![];
// 모든 파일의 정의들을 병합
```

```
for (file_id, definitions) in program_contents {
if let Err(mut errs) = merger.add_definitions(file_id, definitions) {
reports.append(&mut errs);
}
}
let (mut fresh_id, functions, templates, buses) = merger.decompose();
// 빠른 검색을 위한 키 집합 생성
let mut function_keys = HashSet::new();
let mut template_keys = HashSet::new();
let mut bus_keys = HashSet::new();
for key in functions.keys() { function_keys.insert(key.clone()); }
for key in templates.keys() { template_keys.insert(key.clone()); }
for key in buses.keys() { bus_keys.insert(key.clone()); }
// 메인 컴포넌트 처리
let (public_inputs, mut initial_template_call) = main_component;
initial_template_call.fill(file_id_main, &mut fresh_id);
if reports.is_empty() {
Ok(ProgramArchive { /* ... */ })
} else {
Err((file_library, reports))
}
}
```

```
}
```

ProgramArchive 접근 인터페이스

템플릿 관리 메서드들

```
impl ProgramArchive {
pub fn contains_template(&self, template_name: &str) -> bool {
self.templates.contains_key(template_name)
}
pub fn get_template_data(&self, template_name: &str) -> &TemplateData {
assert!(self.contains_template(template_name));
self.templates.get(template_name).unwrap()
}
pub fn get_mut_template_data(&mut self, template_name: &str) -> &mut Template
assert!(self.contains_template(template_name));
self.templates.get_mut(template_name).unwrap()
}
pub fn get_template_names(&self) -> &HashSet<String> {
&self.template_keys
}
pub fn remove_template(&mut self, id: &str) {
self.template_keys.remove(id);
self.templates.remove(id);
```

```
  }

}
```

함수와 버스에 대해서도 동일한 패턴의 메서드들이 제공됩니다.

프로그램 아카이브의 설계 원칙

1. **일관성 보장**: 모든 정의가 고유한 이름을 가지도록 보장
2. **효율적 접근**: 해시맵과 키 집합을 통한 O(1) 검색
3. **메타데이터 완전성**: 모든 AST 노드에 파일 정보 포함
4. **에러 수집**: 병합 과정에서 발생하는 모든 에러를 수집하여 일괄 리포팅
5. **확장성**: 새로운 정의 타입 추가 시 기존 구조 유지

컴파일 파이프라인에서의 역할

ProgramArchive는 컴파일 파이프라인에서 다음과 같은 역할을 수행합니다:

1. **파싱 후 통합**: 개별 파일의 AST들을 하나의 프로그램으로 병합
2. **타입 분석 준비**: 모든 정의에 대한 통합된 뷰 제공
3. **제약 생성 입력**: 템플릿 인스턴스화와 제약 조건 생성의 기반 데이터
4. **에러 리포팅**: 파일 위치 정보를 통한 정확한 에러 메시지 생성

이러한 구조를 통해 Circom 컴파일러는 복잡한 다중 파일 프로젝트를 효율적으로 관리하고, 후속 컴파일 단계에서 필요한 모든 정보를 일관된 형태로 제공할 수 있습니다.